GitHub Actions — Category 2: Actions (Building Blocks)

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Strong Jenkins/CI-CD background assumed. Every topic draws explicit Jenkins comparisons. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → YAML/Code → Gotchas → Connections

---

Table of Contents

- 2.1 What is an Action? — JS, Docker, and Composite Types
- 2.2 Using Marketplace Actions — `uses`, Versioning, Pinning ⚠
- 2.3 Writing a JavaScript Action from Scratch
- 2.4 Writing a Docker-based Action from Scratch
- 2.5 Writing Composite Actions — the Jenkins Shared Library Equivalent
- 2.6 Action Inputs, Outputs, and Passing Data Between Steps
- 2.7 Publishing and Versioning Your Own Actions
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

2.1 What is an Action? — JavaScript, Docker, and Composite Action Types

---

What It Is

An **action** is a self-contained, reusable unit of automation that a workflow step calls via `uses:`. Actions abstract common CI/CD tasks — checkout, setup, build, deploy, notify — so teams don't re-implement the same shell scripting across every workflow.

Think of an action as a **function call** inside your pipeline. You give it inputs, it does work, it returns outputs.

Jenkins equivalent:

- A **Jenkins plugin** (pre-packaged capability added to the server)
- A **Shared Library function** (a Groovy function your Jenkinsfile calls)

The key difference from both: actions are **versioned Git repositories** — not a central plugin registry, not a shared server library. Anyone can publish one, anyone can pin to any commit SHA.

Why It Exists

Problem actions solve: Without actions, every team would write the same bash scripts to set up Node, authenticate to AWS, build Docker images, or post Slack notifications. That's copy-paste engineering at scale.

Where Jenkins falls short:

- Jenkins plugins live in a centralized registry, require server-side installation, and frequently have version-conflict hell between plugins.
- Jenkins Shared Libraries are org-internal and require a Groovy runtime on the agent.
- Neither has a clean input/output contract with type safety.

Actions solve this by being:

- **Portable** — any repo can use any action with one `uses:` line
- **Versioned** — pinned at the caller level, not server level
- **Self-describing** — the `action.yml` metadata file declares inputs, outputs, and the runtime
- **Composable** — actions call other actions

How It Works Internally

When the runner encounters a `uses:` step:

```
Runner encounters: uses: actions/checkout@v4
          │
          ▼
1. Parse `owner/repo@ref` format
          │
          ▼
2. Download action repository at that ref
   (cached in RUNNER_TOOL_CACHE between steps of same job)
          │
          ▼
3. Read action.yml / action.yaml metadata
          │
          ▼
4. Determine action type from `runs.using`:
    ┌──────────────┬──────────────────────────────────────┐
    │ node20       │ Execute dist/index.js with bundled Node │
    │ docker       │ Build/pull image, run container         │
    │ composite    │ Expand steps inline on same runner      │
    └──────────────┴──────────────────────────────────────┘
          │
          ▼
5. Map `with:` inputs → action's declared inputs
          │
          ▼
6. Execute action, capture outputs
          │
          ▼
7. Make outputs available via steps.<id>.outputs.<name>
```

Where are actions stored locally on the runner?

```
/home/runner/work/_actions/
  actions/
    checkout/
      v4/              ← downloaded action repo at that ref
        action.yml
        dist/
          index.js
```

The Three Action Types — Side by Side

Dimension	JavaScript Action	Docker Action	Composite Action
Runtime	Node.js (node20) on runner	Docker container	Runner shell directly
Language	JavaScript / TypeScript	Any language	Bash / other shells
Startup speed	Fast (~1s)	Slow (image pull + start)	Fastest (no overhead)
Environment isolation	Shares runner env	Full container isolation	Shares runner env
Can call other actions	Via JS SDK only	No	Yes — `uses:` in steps
Cross-platform	Yes (Node runs everywhere)	Linux only on GH-hosted	Yes
Best for	Complex logic, API calls	Language-specific tools, CLI wrappers	Reusable step sequences
Requires build step	Yes (`npm run build` → `dist/`)	No	No
Jenkins equivalent	Shared Library Groovy function	Docker agent with tool	Shared Library pipeline template

---

Key Components of Every Action

Every action requires exactly **one metadata file** at the root of the action repository:

```
my-action/
├── action.yml        ← REQUIRED: metadata, inputs, outputs, runtime
├── README.md         ← Best practice: docs for marketplace
└── ...               ← Action-type-specific files
```

The `action.yml` schema:

```yaml
name: 'Human-readable Action Name'
description: 'What this action does — shown on Marketplace'
author: 'Your Name or Org'

branding:                   # Marketplace icon and color
  icon: 'upload-cloud'      # Any Feather icon name
  color: 'blue'             # blue, green, red, orange, yellow, purple,
gray-dark

inputs:
  my-input:
    description: 'What this input does'
    required: true          # or false
    default: 'default-val'  # Only for non-required inputs

outputs:
  my-output:
    description: 'What this output contains'
    value: ${{ steps.some-step.outputs.value }}  # For composite only

runs:
  using: 'node20'           # 'node20', 'docker', or 'composite'
  main: 'dist/index.js'     # For JS actions
  # OR:
  # image: 'Dockerfile'     # For Docker actions
  # steps: [...]            # For composite actions
```

Interview Answer (30–45 seconds)

> "An action is the reusable unit of automation in GitHub Actions — what you call with `uses:` in a workflow step. There are three types: JavaScript actions run Node.js code directly on the runner and are fastest; Docker actions spin up a container for full environment isolation and language flexibility; and composite actions chain multiple `run` and `uses` steps together into a reusable sequence without any runtime overhead. Every action is a Git repository with an `action.yml` metadata file declaring its inputs, outputs, and runtime type. Unlike Jenkins plugins that require server-side installation, actions are just versioned repos — you reference them by `owner/repo@ref` and pin to a commit SHA for security."

Deep Dive: Action Resolution

Three ways to reference an action:

```yaml
# 1. Public action from GitHub Marketplace or any public repo
- uses: actions/checkout@v4
- uses: docker/build-push-action@v5
```

```yaml
# 2. Action in the SAME repository (local action)
- uses: ./.github/actions/my-action    # relative to repo root

# 3. Action in a DIFFERENT private repo (same org)
- uses: my-org/private-actions/deploy@v2
# Requires the workflow's GITHUB_TOKEN to have access,
# or use a PAT with repo access
```

Action resolution order for `owner/repo/path@ref`:

1. GitHub resolves `ref` (tag, branch, or SHA) against the action repo
2. Downloads the repo tree at that ref
3. Looks for `action.yml` or `action.yaml` at `path` (default: repo root)
4. Reads `runs.using` to determine execution model
5. Executes accordingly

Common Interview Questions

Q: When would you write a custom action vs just using `run:` shell commands?

> "I'd write a custom action when logic is needed across multiple workflows or repos, when I need typed inputs with validation and defaults, when I want to encapsulate a complex multi-step sequence, or when I need cross-platform behavior. For one-off logic specific to a single workflow, `run:` is fine and simpler. The decision point is reusability and the need for a clean input/output contract."

Q: What's the difference between an action and a reusable workflow?

> "An action is called at the **step** level — it runs within a job on that job's runner. A reusable workflow is called at the **job** level — it creates its own jobs with their own runners. Composite actions can't define services, matrix strategies, or target environments. Reusable workflows can. If you need to reuse a sequence of steps, use a composite action. If you need to reuse an entire job — with its own runner, environment, concurrency, or matrix — use a reusable workflow."

Gotchas ⚠

- **`action.yml` must be at the path you reference.** If your action is at `my-org/actions/deploy/action.yml`, you reference it as `my-org/actions/deploy@ref`, not `my-org/actions@ref`.
- **JavaScript actions MUST be committed with `dist/` bundled.** The runner does not run `npm install` on the action's behalf. You build locally and commit `dist/index.js`.
- **Docker actions only run on Linux runners.** macOS and Windows GitHub-hosted runners don't support Docker action containers (Docker actions can't run on them).
- **Local actions (`./.github/actions/`) require `actions/checkout` to run first.** The runner needs the repo checked out to find local action files.

- **Composite actions cannot use `continue-on-error` at the action level** — only at individual step level within the composite.

2.2 Using Marketplace Actions — `uses`, Versioning, and Pinning ⚠

What It Is

The `uses:` keyword in a step references a pre-built action by its location and version reference. The **Actions Marketplace** is GitHub's public catalog of community and first-party actions.

Jenkins equivalent: Installing a Jenkins plugin from the Plugin Manager, or referencing a Shared Library with a version tag. But with `uses:`, every workflow independently declares and pins its own versions — there's no server-level installation.

Why Pinning Matters — The Security Threat Model

Tags like `@v4` are **mutable references**. Any of these scenarios can silently change what code runs in your pipeline:

```
Scenario 1: Maintainer compromise
  → Attacker gains control of action repo
  → Force-pushes malicious code to v4 tag
  → Your next workflow run executes attacker code with your secrets

Scenario 2: Accidental breakage
  → Maintainer releases v4.1.0, pushes breaking change to @v4 tag
  → Your workflow breaks with no code change on your side

Scenario 3: Supply chain attack
  → Attacker submits malicious dependency update to the action's npm
packages
  → action@v4 now contains compromised dependency
```

SHA pinning eliminates all three scenarios:

```yaml
# ❌ RISKY — mutable tag, no guarantee of content
- uses: actions/checkout@v4

# ❌ RISKY — branch tip changes any time
- uses: some-org/some-action@main

# ❌ RISKY — latest is always moving
- uses: some-org/some-action@latest
```

```
# ✅ SECURE — immutable commit SHA
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2

# ✅ SECURE — SHA with human-readable comment
- uses: docker/build-push-action@4f58ea79222b3b9dc2c8bbdd6debcef730109a75
# v6.9.0
```

How to Find SHA for a Tag

```
# Method 1: GitHub CLI
gh api repos/actions/checkout/git/ref/tags/v4.2.2 --jq '.object.sha'
# If it's an annotated tag, follow to the commit:
gh api repos/actions/checkout/git/tags/<tag-sha> --jq '.object.sha'

# Method 2: GitHub web UI
# Go to: https://github.com/actions/checkout/releases/tag/v4.2.2
# Click the short SHA next to the tag → shows full commit SHA

# Method 3: git CLI
git ls-remote https://github.com/actions/checkout refs/tags/v4.2.2

# Method 4: Use a tool like pin-github-action or Dependabot
```

Automating Pin Updates with Dependabot

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
    groups:
      actions:
        patterns:
          - '*'
    labels:
      - dependencies
      - github-actions
```

Dependabot will automatically open PRs to update pinned SHAs when action maintainers release new versions, including the version tag in the comment.

`uses` Syntax Variants

```yaml
steps:
  # Public action — most common
  - uses: actions/checkout@v4

  # Public action at specific path within the repo
  - uses: actions/aws-actions/amazon-ecr-login@v2
  # (Where amazon-ecr-login is a subdirectory with its own action.yml)

  # Local action — must checkout first
  - uses: ./.github/actions/setup-environment

  # Local action in subdirectory
  - uses: ./.github/actions/deploy/kubernetes

  # Docker Hub image directly (no action.yml needed)
  - uses: docker://alpine:3.18

  # GitHub Container Registry image
  - uses: docker://ghcr.io/my-org/my-action:v1.2.3
```

Action Versioning Conventions

Action authors typically maintain **floating major version tags**:

```
v4           → always points to latest 4.x.x (MUTABLE)
v4.2         → always points to latest 4.2.x (MUTABLE)
v4.2.2       → typically stable after release (USUALLY immutable)
SHA          → guaranteed immutable
```

How well-maintained actions manage tags:

```bash
# Release process for action maintainers
git tag -fa v4 -m "Update v4 tag to v4.2.2"    # Move floating major tag
git tag -a v4.2.2 -m "Release v4.2.2"           # Create specific version
tag
git push origin v4 --force                       # Force-push the floating
tag
git push origin v4.2.2
```

Real-World Production Pattern: Approved Action Allowlist

Large orgs often maintain an allowlist of approved actions with pinned SHAs:

```yaml
# .github/workflows/reusable-approved-actions.yml
# A composite action that other teams must use instead of raw marketplace
actions

name: 'Org-Approved Checkout'
description: 'Pinned and audited checkout action'
inputs:
  fetch-depth:
    default: '1'
runs:
  using: composite
  steps:
    - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2 — audited 2024-01-15
      with:
        fetch-depth: ${{ inputs.fetch-depth }}
```

```yaml
# Org policy: enforce only approved actions via required workflow
# Org Settings → Actions → Policies → Allow selected actions
# Specify: actions/*, aws-actions/*, my-org/*
```

Interview Answer (30–45 seconds)

> "The `uses:` key references a GitHub action by `owner/repo@ref`. The ref can be a branch, tag, or commit SHA. In production, you should always pin to a commit SHA rather than a floating tag like `@v4`, because tags are mutable — a compromised maintainer or accidental breaking change can silently change what code runs in your workflow with your secrets attached. You keep the human-readable version in a comment. For ongoing updates, Dependabot can automatically open PRs to bump pinned SHAs when new versions are released. For enterprise control, GitHub allows org-level policies to restrict which actions workflows can use."

Gotchas ⚠

- **Annotated tags vs lightweight tags:** When you get the SHA of an annotated tag, you get the tag object's SHA, not the commit SHA. You must dereference it to get the commit SHA — otherwise the pin won't work. Use `git ls-remote` or the GitHub API `/git/tags/{sha}` to follow the reference chain.
- **Actions from the same org's private repo need explicit token access.** `GITHUB_TOKEN` can read actions from the same repo, but not from other repos in the org unless they're public or the token has `contents: read` scope on those repos.
- **`docker://` images are not versioned the same way.** Docker image tags are mutable. Use digest pinning: `docker://alpine@sha256:abc123...` for equivalent security.
- **Required workflows (GitHub Enterprise) override per-repo `permissions` settings.** Know this if you're in an enterprise context where org-level policies apply.

- **The Actions Marketplace is NOT curated for security.** Any GitHub user can publish an action. "Verified creator" badge means GitHub confirmed their identity, not that the code is secure.

2.3 Writing a JavaScript Action from Scratch

What It Is

A JavaScript action runs Node.js code directly on the runner without any container overhead. It uses GitHub's official `@actions/core`, `@actions/github`, and `@actions/exec` toolkit packages for interacting with the runner environment, the GitHub API, and executing commands.

Jenkins equivalent: Writing a Groovy function in a Jenkins Shared Library — procedural code that interacts with the pipeline environment via a provided API (`steps.sh()`, `steps.withCredentials()`, etc.).

Why JavaScript Actions?

Pros:

- Fastest execution (no container pull/start)
- Full access to npm ecosystem
- Can run on all runner OS types (Linux, Windows, macOS)
- Official GitHub toolkit for clean API access
- Straightforward testing with Jest

Cons:

- Must commit compiled/bundled `dist/` — source changes require a build step
- Constrained to Node.js runtime
- Not suitable for heavy system tool usage (prefer Docker action)

When to choose JS over Docker:

- You need to call REST APIs (GitHub API, Slack, PagerDuty, etc.)
- You need to parse and transform JSON/YAML
- You need to read/write files programmatically
- You want cross-platform compatibility

How JavaScript Actions Work Internally

```
Workflow step: uses: my-org/my-js-action@v1

Runner execution:
1. Download action repo at v1
2. Read action.yml → runs.using: 'node20'
3. Locate main: 'dist/index.js'
```

```
4. Execute: /opt/hostedtoolcache/node/20.x/bin/node dist/index.js
5. Inputs injected as: INPUT_<NAME> environment variables
   e.g., input 'api-key' → process.env.INPUT_API_KEY
6. Outputs set by writing to GITHUB_OUTPUT env file
7. Exit code 0 = success, non-zero = failure
```

Complete Project Structure

```
my-js-action/
├── action.yml          ← Metadata
├── src/
│   └── index.ts        ← Source (TypeScript)
├── dist/
│   └── index.js        ← Bundled output — MUST be committed
├── __tests__/
│   └── index.test.ts   ← Unit tests
├── package.json
├── tsconfig.json
├── .ncc/               ← Build cache (gitignore)
└── node_modules/       ← (gitignore)
```

Step-by-Step: Building a Real JS Action

1. Initialize the Project

```
mkdir notify-slack-action && cd notify-slack-action
npm init -y

# GitHub Actions toolkit packages
npm install @actions/core @actions/github @actions/http-client

# Build tooling
npm install --save-dev @vercel/ncc typescript @types/node jest ts-jest
```

2. Write `action.yml`

```
# action.yml
name: 'Slack Deploy Notification'
description: 'Sends a deployment status notification to a Slack channel'
author: 'platform-team'

branding:
  icon: 'bell'
  color: 'green'
```

```yaml
inputs:
  slack-webhook-url:
    description: 'Slack Incoming Webhook URL'
    required: true
  status:
    description: 'Deployment status: success, failure, or cancelled'
    required: true
  service-name:
    description: 'Name of the service being deployed'
    required: true
  environment:
    description: 'Target environment (staging, production, etc.)'
    required: false
    default: 'unknown'
  run-url:
    description: 'URL to the GitHub Actions run'
    required: false
    default: ''

outputs:
  message-ts:
    description: 'Slack message timestamp — use to update/delete the
message'

runs:
  using: 'node20'
  main: 'dist/index.js'
```

3. Write the Action Logic (`src/index.ts`)

```typescript
import * as core from '@actions/core';
import { HttpClient } from '@actions/http-client';

// Status emoji and color mapping
const STATUS_CONFIG = {
  success: { emoji: '✅', color: '#36a64f', text: 'Succeeded' },
  failure: { emoji: '❌', color: '#dc3545', text: 'Failed' },
  cancelled: { emoji: '⚠', color: '#ffc107', text: 'Cancelled' },
} as const;

type Status = keyof typeof STATUS_CONFIG;

interface SlackBlock {
  type: string;
  text?: { type: string; text: string };
  color?: string;
  fields?: Array<{ type: string; text: string }>;
}

async function run(): Promise<void> {
```

```
  try {
    // ── 1. Read inputs ──────────────────────────────────────────
    const webhookUrl = core.getInput('slack-webhook-url', { required: true
});
    const status = core.getInput('status', { required: true }) as Status;
    const serviceName = core.getInput('service-name', { required: true });
    const environment = core.getInput('environment');
    const runUrl = core.getInput('run-url');

    // ── 2. Validate inputs ──────────────────────────────────────
    if (!Object.keys(STATUS_CONFIG).includes(status)) {
      throw new Error(`Invalid status '${status}'. Must be: success,
failure, cancelled`);
    }

    const config = STATUS_CONFIG[status];

    // ── 3. Build Slack message payload ──────────────────────────
    const payload = {
      attachments: [
        {
          color: config.color,
          blocks: [
            {
              type: 'section',
              text: {
                type: 'mrkdwn',
                text: `${config.emoji} *Deployment
${config.text}*\n*Service:* \`${serviceName}\``,
              },
            },
            {
              type: 'section',
              fields: [
                { type: 'mrkdwn', text: `*Environment:*\n${environment}`
},
                { type: 'mrkdwn', text: `*Status:*\n${config.text}` },
              ],
            },
            ...(runUrl
              ? [
                  {
                    type: 'actions',
                    elements: [
                      {
                        type: 'button',
                        text: { type: 'plain_text', text: 'View Run' },
                        url: runUrl,
                      },
                    ],
                  },
                ]
              : []),
          ],
```

```
        },
      ],
    };

    // ── 4. Post to Slack ──────────────────────────────────────────
    core.info(`Sending ${status} notification for ${serviceName} to
Slack...`);

    const client = new HttpClient('notify-slack-action/1.0');
    const response = await client.postJson<{ ts: string }>(webhookUrl,
payload);

    if (response.statusCode !== 200) {
      throw new Error(`Slack API returned status ${response.statusCode}`);
    }

    // ── 5. Set outputs ────────────────────────────────────────────
    const messageTs = response.result?.ts ?? '';
    core.setOutput('message-ts', messageTs);

    core.info(`✅ Notification sent. Message TS: ${messageTs}`);

  } catch (error) {
    // Fail the action with a clear error message
    if (error instanceof Error) {
      core.setFailed(error.message);
    } else {
      core.setFailed('An unknown error occurred');
    }
  }
}

// Entry point
run();
```

4. Core Toolkit Functions Reference

```
import * as core from '@actions/core';
import * as github from '@actions/github';
import * as exec from '@actions/exec';

// ── Input / Output ────────────────────────────────────────────────
const val = core.getInput('my-input');                    // Returns
string
const val2 = core.getInput('my-input', { required: true }); // Throws if
missing
const bool = core.getBooleanInput('dry-run');             //
'true'/'false' → boolean
const multi = core.getMultilineInput('files');            // Newline-
separated → string[]
```

```
core.setOutput('key', 'value');                          // Write to
GITHUB_OUTPUT

// ── Logging ──────────────────────────────────────────────────────
core.debug('Only shown with ACTIONS_RUNNER_DEBUG=true');
core.info('Normal log line');
core.notice('Appears as annotation on the commit');
core.warning('Yellow warning annotation');
core.error('Red error annotation');

// Grouping (collapsible in UI)
core.startGroup('Installing dependencies');
// ... logs here are collapsible
core.endGroup();

// ── Failure / State ──────────────────────────────────────────────
core.setFailed('Error message');    // Sets exit code 1 AND logs error
process.exitCode = 1;               // Alternative: just set exit code

// ── Secret Masking ───────────────────────────────────────────────
core.setSecret('dynamic-sensitive-value');  // Masks in all future log
output

// ── Path & Env Manipulation ──────────────────────────────────────
core.addPath('/usr/local/my-tool/bin');    // Adds to PATH for subsequent
steps
core.exportVariable('MY_VAR', 'value');   // Sets env var for subsequent
steps

// ── GitHub Context ───────────────────────────────────────────────
const context = github.context;
console.log(context.repo);        // { owner: 'org', repo: 'repo' }
console.log(context.sha);         // commit SHA
console.log(context.ref);         // 'refs/heads/main'
console.log(context.actor);       // username
console.log(context.eventName);   // 'push', 'pull_request', etc.
console.log(context.payload);     // Full event payload

// ── GitHub API Client ────────────────────────────────────────────
const token = core.getInput('github-token');
const octokit = github.getOctokit(token);

// Create a PR comment
await octokit.rest.issues.createComment({
  ...context.repo,
  issue_number: context.payload.pull_request!.number,
  body: '## Build Results\n✅ All checks passed',
});

// ── Executing Shell Commands ─────────────────────────────────────
import * as exec from '@actions/exec';

const exitCode = await exec.exec('npm', ['test']);
// With output capture:
```

```
let output = '';
await exec.exec('git', ['log', '--oneline', '-5'], {
  listeners: {
    stdout: (data) => { output += data.toString(); }
  }
});
```

5. Build and Package

```
# Build to dist/ using ncc (bundles all node_modules into single file)
npx @vercel/ncc build src/index.ts --license licenses.txt

# Output: dist/index.js (self-contained, no node_modules needed)
# MUST commit dist/ to the action repo

# package.json scripts
{
  "scripts": {
    "build": "ncc build src/index.ts -o dist --source-map --license
licenses.txt",
    "test": "jest",
    "package": "npm run build && git add dist -f"
  }
}
```

6. Writing Tests

```
// __tests__/index.test.ts
import * as core from '@actions/core';

// Mock the @actions/core module
jest.mock('@actions/core');

describe('notify-slack-action', () => {
  const mockGetInput = core.getInput as jest.MockedFunction<typeof
core.getInput>;
  const mockSetOutput = core.setOutput as jest.MockedFunction<typeof
core.setOutput>;
  const mockSetFailed = core.setFailed as jest.MockedFunction<typeof
core.setFailed>;

  beforeEach(() => {
    jest.clearAllMocks();
  });

  it('should fail with invalid status', async () => {
    mockGetInput.mockImplementation((name) => {
      if (name === 'status') return 'invalid-status';
      if (name === 'service-name') return 'my-api';
```

```
      return '';
    });

    const { run } = await import('../src/index');
    await run();

    expect(mockSetFailed).toHaveBeenCalledWith(
      expect.stringContaining('Invalid status')
    );
  });
});
```

7. Usage in a Workflow

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy application
        id: deploy
        run: ./scripts/deploy.sh

      - name: Notify Slack on success
        if: success()
        uses: my-org/notify-slack-action@abc123def456  # pinned SHA
        with:
          slack-webhook-url: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}
          status: success
          service-name: payment-api
          environment: production
          run-url: ${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}

      - name: Notify Slack on failure
        if: failure()
        uses: my-org/notify-slack-action@abc123def456
        with:
          slack-webhook-url: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}
          status: failure
          service-name: payment-api
          environment: production
```

Interview Answer (30–45 seconds)

> "A JavaScript action uses the Node.js runtime on the runner and GitHub's official `@actions/core` toolkit for reading inputs, setting outputs, logging, and interacting with the GitHub API. The source code is compiled and bundled — using `ncc` — into a single `dist/index.js` file that must be

> committed to the action repo, because the runner doesn't run `npm install` for you. Inputs are injected as `INPUT_<NAME>` environment variables and read with `core.getInput()`. Outputs are written to `$GITHUB_OUTPUT` via `core.setOutput()`. Failures are signaled with `core.setFailed()` which sets a non-zero exit code. The big advantage is speed and cross-platform compatibility — no Docker overhead, runs on Linux, Windows, and macOS."

Gotchas ⚠

- **`dist/` must be committed.** If you forget to rebuild and commit `dist/` after changing source, the action runs stale code. CI should enforce: build → commit dist → push.
- **`ncc` bundles node_modules into `dist/index.js`.** This means `node_modules/` should be gitignored from the action repo, but `dist/` must be committed. Confusing but intentional.
- **`core.setFailed()` does NOT immediately stop execution.** It marks the step as failed and sets exit code 1 at process exit, but code after `setFailed()` still runs. Use `return` after `setFailed()` or rely on the try/catch pattern shown above.
- **`core.getInput()` returns empty string for unset optional inputs**, not `undefined` or `null`. Check `if (!val)` rather than `if (val === undefined)`.
- **The `github.context.payload` shape varies by event type.** A push payload differs from a PR payload. Always check `context.eventName` before accessing event-specific fields.
- **Node.js version matters.** `node20` is current. `node16` is deprecated. Don't use `node12` — it's removed from hosted runners.

2.4 Writing a Docker-based Action from Scratch

What It Is

A Docker action runs inside a container. You define either a `Dockerfile` to build from, or a pre-built image from a registry. The container is the execution environment — completely isolated from the runner's OS.

Jenkins equivalent: Using a Jenkins `agent { docker { image '...' } }` block — the job runs inside a Docker container pulled at build time. Same concept, just configured at the action level instead of the pipeline level.

Why Docker Actions?

Pros:

- Any language: Python, Ruby, Go, Rust, bash — anything that runs in a container
- Consistent environment regardless of runner OS
- Full control over system dependencies
- No build step to commit — the Dockerfile IS the source

Cons:

- Slower: Docker daemon must pull/build the image on each run
- Linux only on GitHub-hosted runners
- More complex than composite or JS for simple tasks
- Image size matters — large images = long startup times

When to choose Docker over JS:

- You need specific system libraries (`libssl`, `libxml2`, tools that aren't in apt)
- You're wrapping an existing CLI tool (e.g., Trivy, Snyk, custom internal tool)
- You're working in a language other than JavaScript
- You need an exact, reproducible binary environment

How Docker Actions Work Internally

```
Workflow step: uses: my-org/trivy-scan-action@v1

Runner execution:
1. Download action repo
2. Read action.yml → runs.using: 'docker'
3. Option A: image: 'Dockerfile'
   → Build Dockerfile in action repo context
   → docker build -t action-image .

   Option B: image: 'docker://aquasec/trivy:0.48.0'
   → docker pull aquasec/trivy:0.48.0

4. docker run \
   --workdir /github/workspace \
   --rm \
   -e INPUT_<NAME>=<value> \     ← inputs as env vars
   -e GITHUB_* \                 ← all GITHUB_ vars forwarded
   -v /home/runner/work:/github/workspace \   ← workspace mounted
   -v /home/runner/_temp/_github_home:/github/home \
   action-image \
   [args from action.yml]

5. Container writes outputs to /github/output → mapped to GITHUB_OUTPUT
6. Container exit code → step success/failure
```

Complete Example: Security Scanner Docker Action

Directory Structure

```
trivy-scan-action/
├── action.yml
├── Dockerfile
```

```
├── entrypoint.sh
└── README.md
```

action.yml

```yaml
name: 'Trivy Security Scanner'
description: 'Scans container images or filesystems for vulnerabilities
using Trivy'
author: 'platform-security-team'

branding:
  icon: 'shield'
  color: 'red'

inputs:
  scan-type:
    description: 'Type of scan: image, fs, repo'
    required: true
    default: 'fs'
  target:
    description: 'Scan target: image name, directory path, or git repo
URL'
    required: true
  severity:
    description: 'Comma-separated severity levels to report'
    required: false
    default: 'CRITICAL,HIGH'
  exit-code:
    description: 'Exit code when vulnerabilities are found'
    required: false
    default: '1'
  format:
    description: 'Output format: table, json, sarif, cyclonedx'
    required: false
    default: 'table'
  output-file:
    description: 'Write results to this file (optional)'
    required: false
    default: ''
  ignore-unfixed:
    description: 'Ignore vulnerabilities with no available fix'
    required: false
    default: 'false'

outputs:
  vulnerability-count:
    description: 'Number of vulnerabilities found at specified severity'
  sarif-file:
    description: 'Path to SARIF output file (if format=sarif)'

runs:
```

```
    using: 'docker'
    image: 'Dockerfile'
    # Alternative: use pre-built image
    # image: 'docker://aquasec/trivy:0.48.0'
    args:
      - ${{ inputs.scan-type }}
      - ${{ inputs.target }}
```

Dockerfile

```
# Use official Trivy image as base — specific version, not latest
FROM aquasec/trivy:0.48.0

# Install jq for JSON processing in entrypoint
RUN apk add --no-cache jq bash

# Copy entrypoint script
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

# GitHub Actions Docker actions run as root by default
# If your action needs a non-root user:
# RUN adduser -D appuser
# USER appuser

ENTRYPOINT ["/entrypoint.sh"]
```

entrypoint.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ── Read inputs from environment variables ──────────────────────────
SCAN_TYPE="${INPUT_SCAN_TYPE:-fs}"
TARGET="${INPUT_TARGET:-.}"
SEVERITY="${INPUT_SEVERITY:-CRITICAL,HIGH}"
EXIT_CODE="${INPUT_EXIT_CODE:-1}"
FORMAT="${INPUT_FORMAT:-table}"
OUTPUT_FILE="${INPUT_OUTPUT_FILE:-}"
IGNORE_UNFIXED="${INPUT_IGNORE_UNFIXED:-false}"

echo "::group::Trivy Scan Configuration"
echo "Scan type:      ${SCAN_TYPE}"
echo "Target:         ${TARGET}"
echo "Severity:       ${SEVERITY}"
echo "Format:         ${FORMAT}"
echo "Ignore unfixed: ${IGNORE_UNFIXED}"
echo "::endgroup::"
```

```
# ── Build Trivy command ──────────────────────────────────────────
TRIVY_ARGS=(
  "${SCAN_TYPE}"
  "--severity" "${SEVERITY}"
  "--exit-code" "${EXIT_CODE}"
  "--format" "${FORMAT}"
  "--no-progress"
)

if [ "${IGNORE_UNFIXED}" = "true" ]; then
  TRIVY_ARGS+=("--ignore-unfixed")
fi

# Handle output file
SARIF_PATH=""
if [ -n "${OUTPUT_FILE}" ]; then
  TRIVY_ARGS+=("--output" "${OUTPUT_FILE}")
  SARIF_PATH="${OUTPUT_FILE}"
elif [ "${FORMAT}" = "sarif" ]; then
  SARIF_PATH="${GITHUB_WORKSPACE}/trivy-results.sarif"
  TRIVY_ARGS+=("--output" "${SARIF_PATH}")
fi

TRIVY_ARGS+=("${TARGET}")

# ── Run Trivy ────────────────────────────────────────────────────
echo "::group::Trivy Scan Results"
set +e   # Don't exit on Trivy finding vulns (it returns exit code from --
exit-code)
trivy "${TRIVY_ARGS[@]}"
TRIVY_EXIT_CODE=$?
set -e
echo "::endgroup::"

# ── Count vulnerabilities ────────────────────────────────────────
VULN_COUNT=0
if [ "${FORMAT}" != "table" ] && [ -n "${SARIF_PATH}" ] && [ -f
"${SARIF_PATH}" ]; then
  VULN_COUNT=$(jq '[.runs[].results | length] | add // 0' "${SARIF_PATH}"
2>/dev/null || echo "0")
fi

# ── Set outputs ──────────────────────────────────────────────────
# In Docker actions, write to $GITHUB_OUTPUT file
echo "vulnerability-count=${VULN_COUNT}" >> "${GITHUB_OUTPUT}"
if [ -n "${SARIF_PATH}" ]; then
  echo "sarif-file=${SARIF_PATH}" >> "${GITHUB_OUTPUT}"
fi

# ── Summary ──────────────────────────────────────────────────────
cat >> "${GITHUB_STEP_SUMMARY}" << EOF
## 🔒 Trivy Security Scan

| Setting | Value |
```

```
|---------|-------|
| Scan Type | \`${SCAN_TYPE}\` |
| Target | \`${TARGET}\` |
| Severity Filter | \`${SEVERITY}\` |
| Vulnerabilities Found | **${VULN_COUNT}** |
EOF

if [ ${TRIVY_EXIT_CODE} -ne 0 ] && [ "${EXIT_CODE}" != "0" ]; then
  echo "" >> "${GITHUB_STEP_SUMMARY}"
  echo "❌ **Scan failed: vulnerabilities at or above ${SEVERITY} severity
found.**" >> "${GITHUB_STEP_SUMMARY}"
fi

exit ${TRIVY_EXIT_CODE}
```

Usage in a Workflow

```yaml
name: Security Scan Pipeline

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
  security-events: write    # Required for uploading SARIF to GitHub
Security

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Scan the filesystem
      - name: Trivy filesystem scan
        id: trivy-fs
        uses: my-org/trivy-scan-action@sha256pinned  # pin to SHA
        with:
          scan-type: fs
          target: .
          severity: CRITICAL,HIGH,MEDIUM
          format: sarif
          ignore-unfixed: true

      # Upload results to GitHub Security tab
      - name: Upload SARIF to GitHub Security
        if: always()
        uses: github/codeql-action/upload-sarif@v3
        with:
```

```yaml
          sarif_file: ${{ steps.trivy-fs.outputs.sarif-file }}

      # Scan the built Docker image
      - name: Build image for scanning
        run: docker build -t myapp:${{ github.sha }} .

      - name: Trivy image scan
        uses: my-org/trivy-scan-action@sha256pinned
        with:
          scan-type: image
          target: myapp:${{ github.sha }}
          severity: CRITICAL
          exit-code: 1    # Fail the build on CRITICAL vulns

      - name: Report vulnerability count
        run: |
          echo "Found ${{ steps.trivy-fs.outputs.vulnerability-count }}
vulnerabilities"
```

Interview Answer (30–45 seconds)

> "A Docker action runs inside a container rather than directly on the runner. You specify either a Dockerfile to build from or a pre-built registry image in `action.yml`. The runner builds or pulls the image, then runs it with the workspace mounted and inputs passed as environment variables named `INPUT_<UPPERCASED-NAME>`. The entrypoint script does the work and writes outputs to the `$GITHUB_OUTPUT` file — same as any other action type. The key advantages are language agnosticism and environment consistency — you can wrap any tool. The tradeoff is startup time from image pull and the Linux-only restriction on GitHub-hosted runners."

Gotchas ⚠

- **Docker actions only run on Linux.** GitHub-hosted macOS and Windows runners don't have Docker available. If you need cross-platform, use JS or Composite.
- **`image: 'Dockerfile'` rebuilds the image on every run.** For actions used frequently, prefer a pre-built image pushed to GHCR and reference it via `docker://ghcr.io/...` with a digest pin.
- **The working directory inside the container is `/github/workspace`**, which maps to `$GITHUB_WORKSPACE` on the runner. Your entrypoint must use this path for accessing repo files.
- **`GITHUB_OUTPUT` path inside the container is `/github/output`** — but the variable `$GITHUB_OUTPUT` is automatically set by the runner and injected. Always use `$GITHUB_OUTPUT` variable, not hardcoded path.
- **Don't run as root unnecessarily.** If your container's tool writes files, they'll be owned by root, which can cause permission issues in subsequent steps on the runner.
- **Image caching is per-runner.** GitHub-hosted runners don't persist Docker layer cache between jobs. Self-hosted runners do. Pre-built images help but you still pay pull time on hosted runners.

2.5 Writing Composite Actions — the Jenkins Shared Library Equivalent

---

What It Is

A composite action chains together multiple `run` steps and `uses` steps into a reusable unit — without any container or Node.js runtime overhead. It runs directly on the caller's runner, using the caller's environment.

Jenkins equivalent: This is the closest thing to a **Jenkins Shared Library pipeline function**. In Shared Libraries you'd define `def deployToK8s(Map config)` and call it from Jenkinsfiles. In GitHub Actions, you define a composite action with `inputs` and call it with `uses:`.

This is the most widely used action type for org-level reusability.

Why Composite Actions?

Pros:

- Zero runtime overhead — runs inline on the caller's runner
- Can call other `uses:` actions (unlike shell scripts)
- Typed inputs and outputs with documentation
- Cross-platform (whatever the caller's runner supports)
- No build step needed — YAML is the source
- Shows as expanded steps in the workflow UI (collapsible)

Cons:

- Cannot define its own services, matrix, or environment
- Cannot set job-level permissions
- No native error handling across steps (use `continue-on-error` per step)
- All steps share the caller's environment (no isolation)

Complete Examples

Example 1: Go Build and Test Setup

```yaml
# .github/actions/setup-go-env/action.yml
name: 'Setup Go Build Environment'
description: 'Installs Go, configures module cache, and downloads
dependencies'
author: 'platform-team'

inputs:
  go-version:
    description: 'Go version to install'
    required: false
```

```yaml
      default: '1.21'
    working-directory:
      description: 'Directory containing go.mod'
      required: false
      default: '.'
    run-tests:
      description: 'Whether to run tests after setup'
      required: false
      default: 'false'
    test-flags:
      description: 'Additional flags to pass to go test'
      required: false
      default: '-race -count=1'

  outputs:
    go-version-installed:
      description: 'The exact Go version that was installed'
      value: ${{ steps.go-setup.outputs.go-version }}
    cache-hit:
      description: 'Whether the module cache was restored'
      value: ${{ steps.cache.outputs.cache-hit }}

  runs:
    using: 'composite'
    steps:
      # Step 1: Install Go
      - name: Setup Go ${{ inputs.go-version }}
        id: go-setup
        uses: actions/setup-go@v5          # Can call other actions!
        with:
          go-version: ${{ inputs.go-version }}
          cache: false                      # We handle caching ourselves

      # Step 2: Cache Go modules
      - name: Cache Go modules
        id: cache
        uses: actions/cache@v4
        with:
          path: |
            ~/go/pkg/mod
            ~/.cache/go-build
          key: ${{ runner.os }}-go-${{ inputs.go-version }}-${{
hashFiles(format('{0}/go.sum', inputs.working-directory)) }}
          restore-keys: |
            ${{ runner.os }}-go-${{ inputs.go-version }}-
            ${{ runner.os }}-go-

      # Step 3: Download dependencies
      - name: Download Go modules
        shell: bash
        working-directory: ${{ inputs.working-directory }}
        run: |
          echo "::group::Downloading Go modules"
          go mod download
```

```
        go mod verify
        echo "::endgroup::"

    # Step 4: Optionally run tests
    - name: Run tests
      if: inputs.run-tests == 'true'
      shell: bash                        # REQUIRED for run steps in
composite
      working-directory: ${{ inputs.working-directory }}
      run: |
        echo "::group::Running Go tests"
        go test ${{ inputs.test-flags }} ./...
        echo "::endgroup::"
```

Note: Every `run:` step in a composite action **must** specify `shell:`. This is a requirement — there's no default shell for composite action steps.

Example 2: Production Kubernetes Deploy Action (SRE Use Case)

```
# .github/actions/k8s-deploy/action.yml
name: 'Kubernetes Helm Deploy'
description: 'Authenticates to EKS, renders Helm chart, deploys, and
verifies rollout'

inputs:
  aws-role-arn:
    description: 'IAM Role ARN to assume (OIDC)'
    required: true
  aws-region:
    description: 'AWS region'
    required: false
    default: 'us-east-1'
  cluster-name:
    description: 'EKS cluster name'
    required: true
  namespace:
    description: 'Kubernetes namespace'
    required: true
  release-name:
    description: 'Helm release name'
    required: true
  chart-path:
    description: 'Path to Helm chart directory'
    required: true
  values-file:
    description: 'Path to values YAML'
    required: false
    default: 'helm/values.yaml'
  image-tag:
    description: 'Docker image tag to deploy'
```

```
      required: true
    rollout-timeout:
      description: 'Timeout for rollout verification'
      required: false
      default: '5m'
    dry-run:
      description: 'Run in dry-run mode (no actual changes)'
      required: false
      default: 'false'

  outputs:
    deployed-image:
      description: 'Full image:tag that was deployed'
      value: ${{ steps.set-outputs.outputs.deployed-image }}
    helm-revision:
      description: 'Helm release revision number after deploy'
      value: ${{ steps.set-outputs.outputs.helm-revision }}

  runs:
    using: 'composite'
    steps:
      # ── Auth ──────────────────────────────────────────────────────
      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ inputs.aws-role-arn }}
          aws-region: ${{ inputs.aws-region }}

      # ── Kubeconfig ────────────────────────────────────────────────
      - name: Update kubeconfig for EKS
        shell: bash
        run: |
          aws eks update-kubeconfig \
            --name ${{ inputs.cluster-name }} \
            --region ${{ inputs.aws-region }}

      # ── Pre-deploy validation ─────────────────────────────────────
      - name: Helm lint
        shell: bash
        run: |
          helm lint ${{ inputs.chart-path }} \
            --values ${{ inputs.values-file }} \
            --set image.tag=${{ inputs.image-tag }}

      - name: Helm diff (show what will change)
        shell: bash
        continue-on-error: true    # Don't fail if helm-diff plugin not
installed
        run: |
          helm diff upgrade \
            ${{ inputs.release-name }} \
            ${{ inputs.chart-path }} \
            --namespace ${{ inputs.namespace }} \
            --values ${{ inputs.values-file }} \
```

```
            --set image.tag=${{ inputs.image-tag }} \
            --allow-unreleased

      # ── Deploy ──────────────────────────────────────────────────────
      - name: Helm deploy
        shell: bash
        run: |
          DRY_RUN_FLAG=""
          if [ "${{ inputs.dry-run }}" = "true" ]; then
            DRY_RUN_FLAG="--dry-run"
            echo "::notice::Running in dry-run mode — no changes will be
made"
          fi

          helm upgrade --install \
            ${{ inputs.release-name }} \
            ${{ inputs.chart-path }} \
            --namespace ${{ inputs.namespace }} \
            --create-namespace \
            --values ${{ inputs.values-file }} \
            --set image.tag=${{ inputs.image-tag }} \
            --wait \
            --timeout ${{ inputs.rollout-timeout }} \
            --atomic \          # Rollback automatically on failure
            ${DRY_RUN_FLAG}

      # ── Verify ──────────────────────────────────────────────────────
      - name: Verify rollout
        if: inputs.dry-run != 'true'
        shell: bash
        run: |
          echo "::group::Rollout status"
          kubectl rollout status deployment/${{ inputs.release-name }} \
            --namespace ${{ inputs.namespace }} \
            --timeout=${{ inputs.rollout-timeout }}
          echo "::endgroup::"

          echo "::group::Pod status after deploy"
          kubectl get pods \
            --namespace ${{ inputs.namespace }} \
            --selector="app.kubernetes.io/name=${{ inputs.release-name }}" \
            --output wide
          echo "::endgroup::"

      # ── Set outputs ─────────────────────────────────────────────────
      - name: Set action outputs
        id: set-outputs
        if: inputs.dry-run != 'true'
        shell: bash
        run: |
          REVISION=$(helm history ${{ inputs.release-name }} \
            --namespace ${{ inputs.namespace }} \
            --max 1 \
            --output json | jq -r '.[0].revision')
```

```
        echo "deployed-image=${{ inputs.image-tag }}" >> $GITHUB_OUTPUT
        echo "helm-revision=${REVISION}" >> $GITHUB_OUTPUT

    # ── Step summary ──────────────────────────────────────────────
    - name: Write deployment summary
      if: always()
      shell: bash
      run: |
        cat >> $GITHUB_STEP_SUMMARY << EOF
        ## 🚀 Kubernetes Deployment

        | Field | Value |
        |-------|-------|
        | Cluster | \`${{ inputs.cluster-name }}\` |
        | Namespace | \`${{ inputs.namespace }}\` |
        | Release | \`${{ inputs.release-name }}\` |
        | Image Tag | \`${{ inputs.image-tag }}\` |
        | Dry Run | \`${{ inputs.dry-run }}\` |
        EOF
```

Usage:

```
jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    environment: staging
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to staging
        id: deploy
        uses: ./.github/actions/k8s-deploy
        with:
          aws-role-arn: ${{ secrets.STAGING_DEPLOY_ROLE }}
          cluster-name: staging-eks
          namespace: my-app
          release-name: payment-api
          chart-path: ./helm/payment-api
          image-tag: ${{ github.sha }}
          dry-run: ${{ github.event_name == 'pull_request' }}

      - name: Log deployment info
        run: |
          echo "Deployed: ${{ steps.deploy.outputs.deployed-image }}"
          echo "Helm revision: ${{ steps.deploy.outputs.helm-revision }}"
```

How Composite Actions Appear in the UI

Unlike a single step, composite action steps are **visible as individual steps** in the workflow run UI, grouped under the action name. This makes debugging much easier than a black-box shell script.

```
✅ Deploy to staging
  ✅ Configure AWS credentials via OIDC
  ✅ Update kubeconfig for EKS
  ✅ Helm lint
  ✅ Helm diff (show what will change)
  ✅ Helm deploy
  ✅ Verify rollout
  ✅ Set action outputs
  ✅ Write deployment summary
```

Interview Answer (30–45 seconds)

> "Composite actions are the closest equivalent to Jenkins Shared Library functions. They chain `run` and `uses` steps together into a reusable action that executes on the caller's runner — no Docker, no Node.js overhead. Every `run` step needs an explicit `shell:` declaration. They support typed inputs and outputs, can call other actions with `uses:`, and their steps are individually visible in the Actions UI which makes debugging transparent. The key limitation compared to reusable workflows is that composite actions can't define their own runners, services, or matrix — they borrow the caller's execution context entirely."

Gotchas ⚠

- **`shell:` is mandatory for every `run:` step in a composite action.** Without it, the step fails. Use `shell: bash` for Linux/macOS, `shell: pwsh` for Windows, or `shell: ${{ runner.os == 'Windows' && 'pwsh' || 'bash' }}` for cross-platform.
- **No `continue-on-error` at the composite action level.** You can use it on individual steps within the composite, but if you want the composite's caller to continue on error, set `continue-on-error: true` on the `uses:` step in the calling workflow.
- **Outputs from composite actions need explicit `value:` references.** Unlike JS actions where you call `core.setOutput()`, composite action outputs must map to a specific step's output: `value: ${{ steps.some-step.outputs.some-output }}`.
- **Environment variables set in composite steps DO propagate to subsequent steps in the CALLER's job** via `$GITHUB_ENV`. This can cause unexpected cross-contamination if not documented.
- **Local composite actions (`./.github/actions/`) must be checked out before use.** The `actions/checkout` step must run before any `uses: ./.github/actions/...` step.
- **Composite actions can be nested** — a composite action can call another composite action. But deeply nested composites make debugging harder.

2.6 Action Inputs, Outputs, and Passing Data Between Steps

What It Is

The formal data contract of an action — how callers pass parameters in (`inputs`) and how actions return results out (`outputs`). This is the equivalent of function arguments and return values.

Inputs — Complete Reference

Declaring Inputs in `action.yml`

```yaml
inputs:
  # Required string input
  image-tag:
    description: 'Docker image tag to deploy'
    required: true

  # Optional string with default
  environment:
    description: 'Target environment'
    required: false
    default: 'staging'

  # Boolean input (represented as string 'true'/'false')
  dry-run:
    description: 'Run without making changes'
    required: false
    default: 'false'

  # Multi-line / list input
  extra-args:
    description: 'Additional arguments (one per line)'
    required: false
    default: ''

  # Numeric input (represented as string)
  timeout-minutes:
    description: 'Job timeout in minutes'
    required: false
    default: '15'
```

Reading Inputs by Action Type

JavaScript Action:

```typescript
import * as core from '@actions/core';

// String input
const imageTag = core.getInput('image-tag', { required: true });

// Boolean input (converts 'true'/'false' string to boolean)
const dryRun = core.getBooleanInput('dry-run');

// Multiline input (splits on newlines, returns string[])
const extraArgs = core.getMultilineInput('extra-args');
const argsStr = extraArgs.join(' ');

// Numeric input
const timeout = parseInt(core.getInput('timeout-minutes'), 10);
```

Docker / Composite Action (shell):

```bash
# Inputs become INPUT_<UPPERCASED_NAME> env vars
# Hyphens become underscores
IMAGE_TAG="${INPUT_IMAGE_TAG}"
DRY_RUN="${INPUT_DRY_RUN}"
TIMEOUT_MINUTES="${INPUT_TIMEOUT_MINUTES}"

# Boolean check
if [ "${INPUT_DRY_RUN}" = "true" ]; then
  echo "Dry run mode enabled"
fi

# Numeric conversion
TIMEOUT=$((INPUT_TIMEOUT_MINUTES * 60))
```

Outputs — Complete Reference

Declaring Outputs in `action.yml`

```yaml
outputs:
  # For JS actions — just declare it
  deployed-version:
    description: 'The version string that was deployed'

  # For composite actions — must include value:
  build-id:
    description: 'Unique build identifier'
    value: ${{ steps.build.outputs.id }}

  release-url:
```

```
    description: 'URL to the GitHub release'
    value: ${{ steps.create-release.outputs.upload_url }}
```

Setting Outputs by Action Type

JavaScript Action:

```
core.setOutput('deployed-version', '1.2.3');
core.setOutput('build-id', buildResult.id.toString());
```

Docker / Shell Action (entrypoint.sh):

```
# Write to $GITHUB_OUTPUT file
echo "deployed-version=1.2.3" >> "${GITHUB_OUTPUT}"
echo "build-id=${BUILD_ID}" >> "${GITHUB_OUTPUT}"

# Multi-line output using heredoc delimiter
REPORT=$(cat report.txt)
EOF=$(dd if=/dev/urandom bs=15 count=1 status=none | base64)
echo "report<<${EOF}" >> "${GITHUB_OUTPUT}"
echo "${REPORT}" >> "${GITHUB_OUTPUT}"
echo "${EOF}" >> "${GITHUB_OUTPUT}"
```

Composite Action:

```
# In a step:
- id: generate-id
  shell: bash
  run: echo "id=$(uuidgen)" >> $GITHUB_OUTPUT

# In outputs declaration:
outputs:
  build-id:
    value: ${{ steps.generate-id.outputs.id }}
```

Data Flow Patterns Across Steps and Jobs

Pattern 1: Step-to-Step Within Same Job

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Get version from package.json
```

```
        id: version         # ← Give the step an ID
        run: |
          VERSION=$(jq -r '.version' package.json)
          echo "value=${VERSION}" >> $GITHUB_OUTPUT
          echo "major=$(echo $VERSION | cut -d. -f1)" >> $GITHUB_OUTPUT

      - name: Build with version
        run: |
          # Reference via: steps.<id>.outputs.<key>
          echo "Building v${{ steps.version.outputs.value }}"
          docker build --build-arg VERSION=${{ steps.version.outputs.value
}} .

      - name: Tag image
        run: |
          docker tag myapp:latest myapp:${{ steps.version.outputs.value }}
          docker tag myapp:latest myapp:${{ steps.version.outputs.major }}
```

Pattern 2: Job-to-Job via Outputs

```
jobs:
  detect-version:
    runs-on: ubuntu-latest
    outputs:
      # Expose step output as job output
      version: ${{ steps.ver.outputs.value }}
      is-release: ${{ steps.ver.outputs.is-release }}
    steps:
      - uses: actions/checkout@v4
      - id: ver
        run: |
          VERSION=$(git describe --tags --always)
          IS_RELEASE=$([[ $VERSION =~ ^v[0-9]+\.[0-9]+\.[0-9]+$ ]] && echo
"true" || echo "false")
          echo "value=${VERSION}" >> $GITHUB_OUTPUT
          echo "is-release=${IS_RELEASE}" >> $GITHUB_OUTPUT

  build:
    needs: detect-version
    runs-on: ubuntu-latest
    steps:
      # Reference via: needs.<job-id>.outputs.<key>
      - run: |
          echo "Building ${{ needs.detect-version.outputs.version }}"
          echo "Is release: ${{ needs.detect-version.outputs.is-release
}}"

  release:
    needs: [detect-version, build]
    if: needs.detect-version.outputs.is-release == 'true'
    runs-on: ubuntu-latest
```

```
    steps:
      - run: echo "Creating release for ${{ needs.detect-
version.outputs.version }}"
```

Pattern 3: Passing Complex Data (JSON) Between Jobs

```
jobs:
  generate-plan:
    runs-on: ubuntu-latest
    outputs:
      deploy-config: ${{ steps.plan.outputs.config }}
    steps:
      - id: plan
        run: |
          # Build a JSON config object
          CONFIG=$(jq -n \
            --arg tag "${{ github.sha }}" \
            --arg env "production" \
            --argjson replicas 3 \
            '{tag: $tag, environment: $env, replicas: $replicas, regions:
["us-east-1","eu-west-1"]}')
          echo "config=${CONFIG}" >> $GITHUB_OUTPUT

  deploy:
    needs: generate-plan
    runs-on: ubuntu-latest
    steps:
      - name: Parse deploy config
        run: |
          CONFIG='${{ needs.generate-plan.outputs.deploy-config }}'
          TAG=$(echo "$CONFIG" | jq -r '.tag')
          ENV=$(echo "$CONFIG" | jq -r '.environment')
          REPLICAS=$(echo "$CONFIG" | jq -r '.replicas')
          echo "Deploying tag=${TAG} to env=${ENV} with
replicas=${REPLICAS}"
```

Pattern 4: Artifacts for Files

When the data is a file (binary, report, build output) rather than a string:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: |
          make build
          make test-report

      - name: Upload test report
```

```yaml
        uses: actions/upload-artifact@v4
        with:
          name: test-results-${{ github.sha }}
          path: |
            coverage/
            test-report.xml
          retention-days: 14

  analyze:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Download test results
        uses: actions/download-artifact@v4
        with:
          name: test-results-${{ github.sha }}
          path: ./results

      - run: ls -la ./results
      - run: cat ./results/test-report.xml
```

Interview Answer (30–45 seconds)

> "Action inputs are declared in `action.yml` with name, description, required flag, and default. They're passed via `with:` in the calling workflow step. In JavaScript actions you read them with `core.getInput()`. In shell-based actions they're injected as `INPUT_<UPPERCASED_NAME>` environment variables. Outputs are set by writing `key=value` to the `$GITHUB_OUTPUT` file — or via `core.setOutput()` in JS actions. Within a job, you access step outputs as `steps.<id>.outputs.<key>`. Across jobs, steps promote values to job-level outputs via the job's `outputs:` block, and downstream jobs access them as `needs.<job>.outputs.<key>`. For files, use upload/download artifact actions since jobs run on separate runners with no shared filesystem."

Gotchas ⚠

- **The old `set-output` command is deprecated and disabled.** `echo "::set-output name=key::value"` no longer works. Always use `>> $GITHUB_OUTPUT`.
- **Output values are always strings.** Even if you write a number or boolean, it comes back as a string. The caller must parse it.
- **Multi-line outputs require a special heredoc format** with a random delimiter to avoid injection:

  ```bash
  EOF=$(openssl rand -base64 12)
  echo "report<<${EOF}" >> $GITHUB_OUTPUT
  echo "line 1" >> $GITHUB_OUTPUT
  echo "line 2" >> $GITHUB_OUTPUT
  echo "${EOF}" >> $GITHUB_OUTPUT
  ```

- **Job outputs only work if the step that sets them actually ran.** If the step is skipped or the job fails before it, `needs.job.outputs.key` will be an empty string, not an error. Build defensive checks with `if:` conditions.
- **`$GITHUB_OUTPUT` is a file, not a socket.** Multiple steps can safely append to it. Don't overwrite it, always append with `>>`.
- **Output size limit is 1 MB per step and 50 KB per individual output value.** For larger data, use artifacts.

2.7 Publishing and Versioning Your Own Actions

What It Is

Making your custom action consumable by other repos and teams — either privately within your org or publicly on the GitHub Marketplace.

Jenkins equivalent: Publishing a Jenkins plugin to the Jenkins Plugin Center, or distributing a Shared Library as a versioned repository. Action publishing is simpler — it's just a Git repository with proper tags.

Versioning Strategy

The convention used by all official GitHub actions:

```
Tags to maintain:
  v2           ← floating major (update on every release)
  v2.1         ← floating minor (update on patch releases)
  v2.1.3       ← immutable specific release

Workflow:
  Release v2.1.3:
    git tag v2.1.3
    git tag -f v2.1           # move floating minor
    git tag -f v2             # move floating major
    git push --tags --force
```

Why maintain floating tags? Callers using `@v2` automatically get bug fixes without changing their workflow files. Callers using `@v2.1.3` get stability. Callers using a SHA get complete immutability.

Complete Release Workflow

```yaml
# .github/workflows/release.yml — in your action repo
name: Release Action

on:
```

```yaml
  push:
    tags:
      - 'v*.*.*'

permissions:
  contents: write

jobs:
  # ── 1. Run tests before releasing ──────────────────────────
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test

  # ── 2. Build and verify ────────────────────────────────────
  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run build   # produces dist/

      # Verify dist is up to date with source
      - name: Verify dist is committed
        run: |
          if [ -n "$(git diff --stat dist/)" ]; then
            echo "::error::dist/ is not up to date. Run 'npm run build'
and commit."
            git diff dist/
            exit 1
          fi

  # ── 3. Create GitHub Release ───────────────────────────────
  release:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0   # Need full history for changelog

      - name: Extract version from tag
        id: version
```

```
          run: |
            TAG="${{ github.ref_name }}"           # e.g., v2.1.3
            MAJOR=$(echo $TAG | cut -d. -f1)       # e.g., v2
            MINOR=$(echo $TAG | cut -d. -f1-2)     # e.g., v2.1
            echo "tag=${TAG}" >> $GITHUB_OUTPUT
            echo "major=${MAJOR}" >> $GITHUB_OUTPUT
            echo "minor=${MINOR}" >> $GITHUB_OUTPUT

        - name: Generate changelog
          id: changelog
          run: |
            # Get commits since last tag
            PREV_TAG=$(git tag --sort=-version:refname | sed -n '2p')
            CHANGES=$(git log ${PREV_TAG}..HEAD --oneline --no-merges)
            EOF=$(openssl rand -base64 12)
            echo "notes<<${EOF}" >> $GITHUB_OUTPUT
            echo "${CHANGES}" >> $GITHUB_OUTPUT
            echo "${EOF}" >> $GITHUB_OUTPUT

        - name: Create GitHub Release
          uses: actions/create-release@v1
          env:
            GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          with:
            tag_name: ${{ steps.version.outputs.tag }}
            release_name: Release ${{ steps.version.outputs.tag }}
            body: |
              ## Changes
              ${{ steps.changelog.outputs.notes }}

              ## Usage
              ```yaml
              - uses: my-org/my-action@${{ steps.version.outputs.tag }}
              ```
            draft: false
            prerelease: false

        # ── 4. Update floating tags ──────────────────────────────
        - name: Update floating major and minor tags
          run: |
            git config user.email "github-
actions[bot]@users.noreply.github.com"
            git config user.name "github-actions[bot]"

            MAJOR="${{ steps.version.outputs.major }}"
            MINOR="${{ steps.version.outputs.minor }}"

            # Move floating tags
            git tag -fa "${MAJOR}" -m "Update ${MAJOR} tag to ${{
steps.version.outputs.tag }}"
            git tag -fa "${MINOR}" -m "Update ${MINOR} tag to ${{
steps.version.outputs.tag }}"
            git push origin "${MAJOR}" --force
            git push origin "${MINOR}" --force
```

Marketplace Publishing

To publish publicly on the GitHub Marketplace:

```
# action.yml requirements for Marketplace listing:
name: 'My Action'                    # Required, must be unique on
Marketplace
description: 'Clear description'    # Required, shown in search results
author: 'Your Name'                  # Recommended

branding:                            # Required for Marketplace
  icon: 'upload-cloud'               # Must be a Feather icon
  color: 'blue'                      # Limited color palette

# When creating a release, check "Publish this release to the GitHub
Marketplace"
# GitHub validates action.yml compliance before allowing listing
```

Private Action Distribution Patterns

Pattern 1: Centralized Action Repo (most common for orgs)

```
my-org/
  platform-actions/           ← All reusable actions live here
    .github/actions/
      setup-go/
        action.yml
      k8s-deploy/
        action.yml
      notify-slack/
        action.yml
    .github/workflows/
      release.yml
    README.md

# Usage in any org repo:
- uses: my-org/platform-actions/.github/actions/k8s-deploy@v3
  with:
    cluster: prod-eks
```

Pattern 2: Actions Co-Located in App Repo

```
my-app/
  .github/
```

```
    actions/
      build/action.yml      ← Local, used only in this repo
      test/action.yml
    workflows/
      ci.yml

# Usage (local reference):
- uses: ./.github/actions/build
```

Pattern 3: Monorepo with Actions Package

```
my-monorepo/
  packages/
    actions/
      setup-env/action.yml
      deploy/action.yml
  apps/
    api/
    frontend/
  .github/
    workflows/
      api-ci.yml            # uses: ./packages/actions/deploy
      frontend-ci.yml
```

Testing Actions Before Release

```yaml
# .github/workflows/ci.yml — test the action itself
name: Test Action

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test-unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test

  test-integration:
    runs-on: ubuntu-latest
```

```yaml
    steps:
      - uses: actions/checkout@v4

      # Test the action itself using a local reference
      - name: Run the action
        id: test-run
        uses: ./                    # ← test current branch's action
        with:
          slack-webhook-url: ${{ secrets.TEST_SLACK_WEBHOOK }}
          status: success
          service-name: test-action-ci
          environment: test

      - name: Verify outputs
        run: |
          if [ -z "${{ steps.test-run.outputs.message-ts }}" ]; then
            echo "::error::Expected message-ts output to be set"
            exit 1
          fi
          echo "✅ message-ts: ${{ steps.test-run.outputs.message-ts }}"

  test-matrix:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: ./
        with:
          status: success
          service-name: cross-platform-test
```

Interview Answer (30–45 seconds)

> "Publishing a custom action is straightforward — it's a Git repo with an `action.yml`. For versioning, you follow the same convention as official actions: maintain floating major version tags like `v2` that you force-update on every release, alongside immutable specific tags like `v2.1.3`. The release workflow builds and verifies `dist/`, creates a GitHub Release with changelog, and uses a git tag force-push to move the floating tags. For org-internal distribution, teams typically keep all platform actions in a centralized repo and reference them cross-repo. For public Marketplace listing, you need proper branding in `action.yml` and check the Marketplace option when creating a release."

Gotchas ⚠

- **Forgetting to rebuild `dist/` before tagging** is the most common release mistake. Always verify `git diff dist/` is clean in your release CI.
- **Force-pushing floating tags requires `contents: write` permission** and `git push --force`. Configure this in your release workflow.

- **Private org actions require the action repo to be accessible.** If the action repo is private, the consuming repo's `GITHUB_TOKEN` must have `contents: read` on it, OR the action repo must be in the same org with appropriate visibility.
- **Marketplace names must be globally unique.** Your `name:` in `action.yml` is checked for uniqueness across all marketplace actions when you try to publish.
- **Semantic release tools** like `semantic-release` or `release-please` can automate the versioning workflow — worth mentioning in senior-level interviews.
- **GitHub scans actions for secrets on publish.** Secret scanning runs on action repos — don't hardcode credentials in action code.

Cross-Topic Interview Questions

These questions span multiple topics in Category 2 and are common in senior-level interviews.

Q: Your team wants to standardize how all services deploy to Kubernetes. You have 30 repos. What's your strategy?

> "I'd build a composite action in a centralized platform-actions repo that encapsulates the full deploy flow — auth via OIDC, kubeconfig setup, Helm deploy, rollout verification, and step summary. The action has typed inputs for cluster name, namespace, image tag, and optional dry-run mode. All 30 repos reference it with `uses: my-org/platform-actions/.github/actions/k8s-deploy@v2`. For the deployment pipeline itself, I'd create a reusable workflow that wraps the action and adds environment protection rules. Teams only need to call the reusable workflow — they don't even need to know about the composite action's internals. When we need to change the deploy process, we update one place and all repos get it on next run since they use the floating `@v2` tag."

Q: How do you prevent a malicious action from stealing your secrets?

> "Multiple layers: first, pin all third-party actions to commit SHAs — not floating tags — so the code we've audited is exactly what runs. Second, set minimal `permissions` at the workflow level and override per-job. Third, use OIDC instead of storing cloud credentials as secrets — OIDC tokens are short-lived and scoped to specific claims. Fourth, use GitHub's org-level action policies to allowlist trusted action sources. Fifth, regularly audit with tools like StepSecurity's Harden-Runner or Dependabot for action version updates. Sixth, never use `pull_request_target` without extreme care since fork PRs can access base-branch secrets. Defense in depth — no single control is sufficient."

Q: When would you choose a Docker action over a composite action?

> "I'd choose Docker when I need a specific binary or system library that's not on the runner — like a particular version of a scanning tool, a custom internal CLI, or a non-JavaScript language runtime. Docker gives complete environment control and repeatability. I'd choose composite when I'm just chaining existing actions or shell commands — it's faster, cross-platform, and simpler to maintain. A

> useful heuristic: if your action is primarily calling other actions and running scripts, composite. If it needs a controlled system environment or a specific tool version locked in, Docker."

Q: A colleague says 'I'll just use `@main` to always get the latest version of an action.' How do you respond?

> "That's a significant security risk. `@main` means every workflow run could execute different code — no predictability, no auditability. If the action repo is compromised or has a bad commit pushed, your pipelines run malicious code with access to all your secrets on the next trigger. The right approach is pinning to a specific commit SHA and using Dependabot to automatically open PRs when the action maintainer releases a new version. That way you get security reviews on updates and maintain an immutable audit trail of exactly what code ran in each pipeline."

Quick Reference Cheat Sheet

Action Type Decision Matrix

```
Need to reuse steps across multiple workflows or repos?
├── Yes → Custom Action
│   ├── Need container isolation or non-JS language? → Docker Action
│   ├── Need complex logic, API calls, cross-platform? → JavaScript Action
│   └── Just chaining steps/actions? → Composite Action ← (most common)
└── No → Use `run:` steps directly in the workflow
```

File Structure at a Glance

```
JS Action:           Docker Action:        Composite Action:
├── action.yml       ├── action.yml        ├── action.yml
├── src/             ├── Dockerfile        └── (no other required files)
│   └── index.ts     └── entrypoint.sh
├── dist/
│   └── index.js     (pre-built image:
└── package.json      docker://registry/img)
```

Input/Output Cheat Sheet

```bash
# Setting outputs (shell)
echo "key=value" >> $GITHUB_OUTPUT

# Multi-line output (shell)
EOF=$(openssl rand -base64 12)
echo "report<<${EOF}" >> $GITHUB_OUTPUT
echo "$CONTENT" >> $GITHUB_OUTPUT
```

```
echo "${EOF}" >> $GITHUB_OUTPUT

# Reading inputs in shell (Docker/composite)
VALUE="${INPUT_MY_INPUT_NAME}"     # hyphens → underscores, uppercase

# Referencing in workflow
steps.<id>.outputs.<key>           # step-to-step
needs.<job>.outputs.<key>          # job-to-job
```

Pinning Commands

```
# Get commit SHA for a tag
git ls-remote https://github.com/actions/checkout refs/tags/v4.2.2

# Via GitHub API (handles annotated tags)
gh api repos/actions/checkout/git/ref/tags/v4.2.2 | jq -r '.object.sha'

# In workflow (secure pattern):
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  #
v4.2.2
```

Composite Action `shell:` Requirements

```
# REQUIRED on every run: step in composite actions
- name: My step
  shell: bash          # Always specify
  run: echo "hello"

# Cross-platform shell selection
  shell: ${{ runner.os == 'Windows' && 'pwsh' || 'bash' }}
```

Common `@actions/core` Methods

```
// Inputs
core.getInput('name')                    // → string
core.getBooleanInput('flag')             // → boolean
core.getMultilineInput('list')           // → string[]

// Outputs
core.setOutput('key', value)

// Logging
core.debug / info / warning / error / notice

// Failure
core.setFailed('message')               // sets exit code 1
```

```
// Env & Path
core.exportVariable('KEY', 'val')       // for subsequent steps
core.addPath('/usr/local/bin')          // adds to PATH

// Secrets
core.setSecret('sensitive-value')       // masks in logs

// Groups
core.startGroup('name') / core.endGroup()
```

End of Category 2: Actions — Building Blocks

> **Next:** Category 3 (Reusability & Modularity) builds directly on these concepts — reusable workflows are the job-level equivalent of composite actions. **Practice:** Try building a composite action that wraps your most common deployment pattern. The muscle memory from writing `action.yml` once makes interview questions on this topic trivial to answer.